

Men's Apparel E-Commerce & Order Management Platform

End-to-End Enterprise Technical Blueprint & Implementation Specification

Production Architecture & Operations Engineering Group

May 22, 2026

Abstract

This document outlines the absolute engineering requirements, transactional protocols, database structures, security boundaries, and user interface directives for building a high-end, responsive, multi-page Men's Dress Shop E-Commerce platform. Engineered with Spring Boot 3.2.x, Java 17, MySQL 8.x, and a Tailwind CSS-driven client interface, this specification represents a complete and rigorous blueprint designed for error-free machine compilation and deployment.

Contents

1	Technical Stack & Multi-Page Architecture	3
1.1	Architectural Pillars	3
1.2	The Multi-Page Directory Matrix	3
1.3	Cross-Page Token Lifecycle & Session Management	3
1.3.1	Silent Refresh Loop Sequence	4
1.4	CORS & CSRF Interplay Policies	4
2	Technical Flows & Operational State Machines	5
2.1	User Registration & Verification OTP Flow	5
2.2	Concurrency-Safe Checkout & Dynamic Invoicing	5
2.2.1	Dynamic Invoice Calculations	5
2.2.2	Inventory Concurrency & Locking Architecture	6
2.2.3	Idempotency Guardrail	6
2.3	Post-Purchase Cancellation & Return Policies	6
2.3.1	The 3-Day Cancellation Window	6
2.3.2	The 1-Day Damaged Product Return Window	6
3	Database Schema Design & Canonical Enums	7
3.1	Canonical Enums Specifications	7
3.2	MySQL DDL Schema Script	7
4	API Integration & Centralized Exception Handling	10
4.1	Centralized Error Schema Representation	10
4.2	API Routes Dictionary	10
5	UI/UX Design Tokens & Animation Guidelines	11
5.1	Design Tokens & Color Palette	11
5.2	Mobile-First Responsiveness & WCAG 2.1 AA Checklist	11
5.3	Valid CSS Keyframes & Premium Animations	11

6	Observability, CI/CD Gating & Operations	13
6.1	Prometheus Metrics & Alerting Targets	13
6.2	CI/CD Compilation Gating Checks	13
6.3	Operations Deployment Checklist	13

1 Technical Stack & Multi-Page Architecture

The system is designed as a secure, distributed Multi-Page Application (MPA) partitioned into three core layers: a persistent storage engine, a stateless security-centric back-end controller, and an accessible, mobile-first client presentation interface.

1.1 Architectural Pillars

Database (MySQL 8.x) The primary datastore. It enforces strict referential integrity, indexes for dynamic analytical reads, explicit column scaling for monetary decimals, and optimistic locking capabilities.

Core API Engine (Spring Boot 3.2.x & Java 17) Organized into modular layers: Config, Entity, Repository, Service, Controller, DTO, and Exception Handlers. Uses Maven for dependency lifecycle operations.

Security Layer Enforces stateless JWT authentication (15-minute access TTL), secure rotated HTTP-only refresh cookies, double-submit cookie Cross-Site Request Forgery (CSRF) defenses, and strict Cross-Origin Resource Sharing (CORS) whitelists.

Client Interface (MPA) Employs seven independent, self-contained HTML pages. Layout structures are driven by Tailwind CSS with custom Vanilla JS API helpers to handle state, JWT-refresh, and local browser cart stores.

1.2 The Multi-Page Directory Matrix

The client must consist of these seven distinct, inter-linked HTML modules:

1. `index.html`: Public catalog entry page featuring search inputs, responsive category-filtering triggers, and item showcase grids.
2. `auth.html`: Unified interface housing the secure Login card, registration forms, and the 6-digit local debug verification OTP panel.
3. `product-details.html`: Responsive item detail layout featuring interactive size matrices, color swatches, rating stars, and live customer comments.
4. `cart.html`: Persistent shopping cart showing chosen clothing variants, editable item counters, and subtotal-calculating matrices.
5. `checkout.html`: Transaction finalization page containing auto-filled address parameters, a secure sandbox Razorpay simulator, and COD choices.
6. `orders.html`: Customer ledger displaying active orders, a visual 4-step progress stepper, 3-day order cancellations, and 1-day damaged item return options.
7. `admin-dashboard.html`: Role-restricted command dashboard displaying analytic metric blocks, master order dispatch grids, and return processing units.

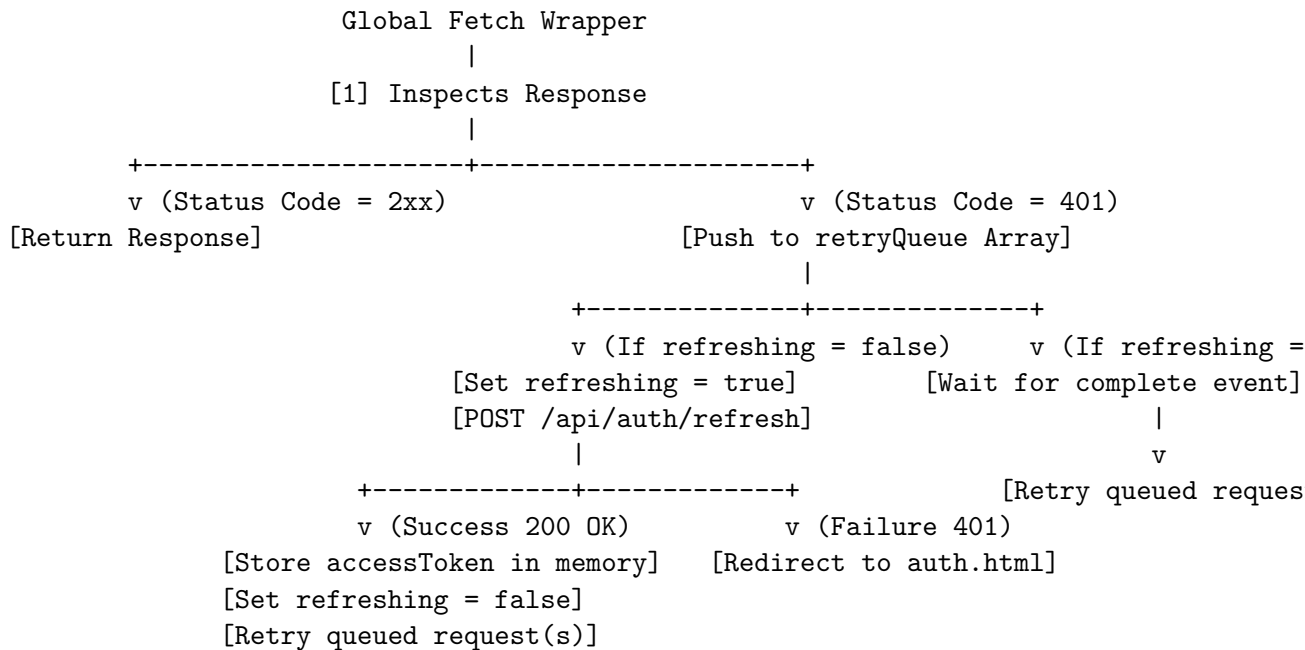
1.3 Cross-Page Token Lifecycle & Session Management

To avoid cross-site scripting (XSS) injection vectors, access tokens must remain strictly in volatile memory. Under no circumstances may access tokens be committed to persistent browser storage.

- **Access Token:** Issued with a strict 15-minute Time-to-Live (TTL). Transmitted via the HTTP standard Authorization: Bearer <accessToken> header.
- **Refresh Token:** Stored inside an encrypted, HttpOnly, Secure, SameSite=Strict cookie. Every refresh event invalidates the current pair and returns a newly rotated token set.

1.3.1 Silent Refresh Loop Sequence

To ensure a continuous customer shopping experience without manual session logouts, all client pages must incorporate the following API interceptor logic:



1.4 CORS & CSRF Interplay Policies

CORS Security The Spring Boot back-end must configure CORS dynamically, permitting only explicitly whitelisted production domains while allowing client headers: Authorization, Content-Type, Idempotency-Key, and X-XSRF-TOKEN.

CSRF Double-Submit Validation Upon session establishment, the back-end issues a secure cookie named XSRF-TOKEN. Before executing any POST, PUT, or DELETE state-changing operations, client-side JavaScript must extract this value from document.cookie and set the custom X-XSRF-TOKEN HTTP request header.

2 Technical Flows & Operational State Machines

This section details the critical operational state engines, constraints, and mathematical bounds governing the platform's core e-commerce transactions.

2.1 User Registration & Verification OTP Flow

The registration-to-login timeline enforces rate-limited user verification before granting workspace permissions:

1. The customer registers. The back-end encrypts the raw password utilizing the BCrypt algorithm (minimum 12 hashing rounds) and persists the record with status: `is_verified = FALSE`.
2. The service generates a random 6-digit numeric verification OTP (C_{otp}), hashes it using HMAC-SHA256 with a secure server-side pepper, and stamps the record with a strict 5-minute expiration block:

$$t_{expiry} = t_{current} + 300 \text{ seconds} \quad (1)$$

3. **Rate Limits & Abuse Safeguards:** A user profile is restricted to 3 OTP generation requests per hour. A 60-second cool-down is enforced between resends. A user is allowed a maximum of 5 verification attempts per generated OTP; exceeding this threshold invalidates the token instantly.
4. **CI Build Gating:** For local sandbox developer testing, the plaintext OTP code may be returned inside a `"debug_otp"` payload field, strictly gated behind a non-production profile (e.g., `spring.profiles.active=dev`). The Continuous Integration (CI) build must execute a static analysis test to fail the compilation if development flags are detected in release branch check-ins.
5. Upon correct OTP submission, matching is performed using constant-time verification blocks. The user is updated to `is_verified = TRUE`, clearing the OTP hashes. Any login attempt from an unverified profile is rejected.

2.2 Concurrency-Safe Checkout & Dynamic Invoicing

Order finalization requires strict thread isolation to prevent inventory anomalies.

2.2.1 Dynamic Invoice Calculations

At checkout, pricing structures are calculated utilizing standard monetary representation types. The total order calculation must strictly apply the following formula:

$$\text{Total Invoice Amount} = \sum_{i=1}^n (\text{Price}_{\text{item } i} \times \text{Quantity}_i) + \text{Sales Tax} + \text{Shipping Cost} \quad (2)$$

Where:

- Sales Tax is set to a constant rate of 12% applied directly to the subtotal.
- Shipping Cost is set to a flat rate of \$15.00, but automatically adjusted to \$0.00 (FREE) if the dynamic order subtotal exceeds \$150.00.

2.2.2 Inventory Concurrency & Locking Architecture

Checkout operations must run within isolation boundaries to avoid overselling.

- **Database Transaction Isolation** Every checkout execution must be isolated under the `@Transactional(isolation = Isolation.REPEATABLE_READ)` block.
- **Optimistic Locking (Default)** The products database catalog must incorporate a `@Version` field (version column). Upon catching an `ObjectOptimisticLockingFailureException`, the application must trigger an automated retry loop. This loop attempts re-execution up to 3 times under exponential back-off delays (first 100ms, then 200ms, then 400ms). If all 3 retries are exhausted or inventory is depleted, the transaction aborts and returns an HTTP 409 Conflict with system code "CONFLICT_STOCK".
- **Pessimistic Locking (High-Write Hotspots)** Under high-volume concurrency conditions, write endpoints are configured to execute pessimistic locks via `@Lock(LockModeType.PESSIMISTIC_WRITE)` mapping to database `SELECT ... FOR UPDATE` actions on affected catalog records.

2.2.3 Idempotency Guardrail

State-changing routes require an Idempotency-Key HTTP header. Key hashes (SHA-256) are committed to a dedicated table with a 24-hour expiration window. Duplicate requests inside this window bypass processing and return the cached payload.

2.3 Post-Purchase Cancellation & Return Policies

The platform enforces temporal boundaries on all order cancellations and return operations.

2.3.1 The 3-Day Cancellation Window

Customers may cancel orders self-service provided that:

$$t_{\text{current}} - t_{\text{order_date}} \leq 259,200 \text{ seconds (3 days)} \quad (3)$$

Upon meeting this constraint, the status updates to CANCELLED in a single transaction, restocking inventory.

- If the payment was completed via RAZORPAY, a simulated gateway refund is triggered. The refund status is updated to REFUNDED and marked with a unique `refund_id` to ensure idempotence.
- If the payment was selected as COD, the payment status is updated directly to CANCELLED.

2.3.2 The 1-Day Damaged Product Return Window

Customers can request returns for damaged items only if the order status is DELIVERED and the request falls within the strict timeline:

$$t_{\text{current}} - t_{\text{actual_delivery_date}} \leq 86,400 \text{ seconds (1 day)} \quad (4)$$

This operation transitions the order to RETURN_REQUESTED. The administrator must approve this request from their dashboard to change the status to RETURNED, restore catalog inventory, and process refunds.

3 Database Schema Design & Canonical Enums

The schema enforces strict relational constraints, precise data mapping, indices for fast lookups, and audit fields to track all transactional changes.

3.1 Canonical Enums Specifications

All database status fields must map strictly to the following uppercase enum strings:

`order_status` ORDER_RECEIVED, DISPATCHED, OUT_FOR_DELIVERY, DELIVERED, CANCELLED, RETURN_REQUESTED, RETURNED.

`payment_status` PENDING_PAY, PAID, COD_COLLECTED, REFUNDED, FAILED, CANCELLED.

`payment_method` RAZORPAY, COD.

3.2 MySQL DDL Schema Script

The following script represents the authoritative database layout for local and production deployment setups.

```
1  -- Initialize Database
2  CREATE DATABASE IF NOT EXISTS mens_dress_shop_db
3      DEFAULT CHARACTER SET utf8mb4
4      DEFAULT COLLATE utf8mb4_unicode_ci;
5  USE mens_dress_shop_db;
6
7  -- Enforce Strict UTC Mapping
8  SET time_zone = '+00:00';
9
10 -- 1. Categories Table
11 CREATE TABLE IF NOT EXISTS categories (
12     id BIGINT AUTO_INCREMENT PRIMARY KEY,
13     name VARCHAR(100) NOT NULL UNIQUE,
14     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
15     updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
16 ) ENGINE=InnoDB;
17
18 -- 2. Products Table
19 CREATE TABLE IF NOT EXISTS products (
20     id BIGINT AUTO_INCREMENT PRIMARY KEY,
21     name VARCHAR(255) NOT NULL,
22     description TEXT NULL,
23     price DECIMAL(10, 2) NOT NULL,
24     image_url VARCHAR(255) NULL,
25     stock_quantity INT NOT NULL DEFAULT 0,
26     sizes VARCHAR(100) NOT NULL DEFAULT 'S,M,L,XL,XXL',
27     colors VARCHAR(100) NOT NULL DEFAULT 'Navy Blue,Jet Black,Charcoal Gray',
28     category_id BIGINT NULL,
29     version INT NOT NULL DEFAULT 0, -- For Optimistic Locking
30     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
31     updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
32     CONSTRAINT fk_prod_cat FOREIGN KEY (category_id) REFERENCES categories(id)
33     ON DELETE SET NULL,
34     CONSTRAINT chk_stock CHECK (stock_quantity >= 0),
35     CONSTRAINT chk_price CHECK (price > 0.00)
36 ) ENGINE=InnoDB;
```

```

36
37 -- 3. Users Table
38 CREATE TABLE IF NOT EXISTS users (
39     id BIGINT AUTO_INCREMENT PRIMARY KEY,
40     email VARCHAR(100) NOT NULL UNIQUE,
41     username VARCHAR(100) NOT NULL UNIQUE,
42     password_hash VARCHAR(255) NOT NULL,
43     role VARCHAR(20) NOT NULL DEFAULT 'CUSTOMER',
44     phone_number VARCHAR(20) NOT NULL,
45     address VARCHAR(255) NOT NULL,
46     city VARCHAR(100) NOT NULL,
47     state VARCHAR(100) NOT NULL,
48     country VARCHAR(100) NOT NULL,
49     date_of_birth DATE NOT NULL,
50     is_verified BOOLEAN NOT NULL DEFAULT FALSE,
51     otp_code_hash VARCHAR(255) NULL,
52     otp_expiry TIMESTAMP NULL,
53     reset_token_hash VARCHAR(255) NULL,
54     reset_token_expiry TIMESTAMP NULL,
55     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
56     updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
57     CONSTRAINT chk_role CHECK (role IN ('CUSTOMER', 'ADMIN'))
58 ) ENGINE=InnoDB;
59
60 -- 4. Orders Table
61 CREATE TABLE IF NOT EXISTS orders (
62     id BIGINT AUTO_INCREMENT PRIMARY KEY,
63     user_id BIGINT NOT NULL,
64     order_date TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
65     total_price DECIMAL(10, 2) NOT NULL,
66     status VARCHAR(50) NOT NULL DEFAULT 'ORDER_RECEIVED',
67     payment_method VARCHAR(50) NOT NULL,
68     payment_status VARCHAR(50) NOT NULL DEFAULT 'PENDING_PAY',
69     payment_id VARCHAR(255) NULL,
70     refund_id VARCHAR(255) NULL,
71     expected_delivery_date DATE NOT NULL,
72     actual_delivery_date TIMESTAMP NULL,
73     version INT NOT NULL DEFAULT 0,
74     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
75     updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
76     CONSTRAINT fk_order_user FOREIGN KEY (user_id) REFERENCES users(id) ON
77         DELETE CASCADE,
78     CONSTRAINT chk_order_status CHECK (status IN ('ORDER_RECEIVED', 'DISPATCHED',
79         'OUT_FOR_DELIVERY', 'DELIVERED', 'CANCELLED', 'RETURN_REQUESTED', '
80         RETURNED')),
81     CONSTRAINT chk_pay_status CHECK (payment_status IN ('PENDING_PAY', 'PAID', '
82         COD_COLLECTED', 'REFUNDED', 'FAILED', 'CANCELLED')),
83     CONSTRAINT chk_pay_method CHECK (payment_method IN ('RAZORPAY', 'COD'))
84 ) ENGINE=InnoDB;
85
86 -- 5. Order Items Table
87 CREATE TABLE IF NOT EXISTS order_items (
88     id BIGINT AUTO_INCREMENT PRIMARY KEY,
89     order_id BIGINT NOT NULL,
90     product_id BIGINT NOT NULL,
91     quantity INT NOT NULL,
92     price_at_purchase DECIMAL(10, 2) NOT NULL,
93     selected_size VARCHAR(10) NOT NULL,

```



```

90     selected_color VARCHAR(50) NOT NULL,
91     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
92     updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
93     CONSTRAINT fk_item_order FOREIGN KEY (order_id) REFERENCES orders(id) ON
        DELETE CASCADE,
94     CONSTRAINT fk_item_prod FOREIGN KEY (product_id) REFERENCES products(id) ON
        DELETE CASCADE,
95     CONSTRAINT chk_qty CHECK (quantity > 0)
96 ) ENGINE=InnoDB;
97
98 -- 6. Idempotent Requests Table
99 CREATE TABLE IF NOT EXISTS idempotent_requests (
100     key_hash CHAR(64) PRIMARY KEY, -- SHA-256 Hex Hash
101     response_payload VARCHAR(10000) NOT NULL,
102     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
103 ) ENGINE=InnoDB;
104
105 -- Optimized Indices
106 CREATE UNIQUE INDEX idx_users_email ON users(email);
107 CREATE UNIQUE INDEX idx_users_username ON users(username);
108 CREATE INDEX idx_orders_user ON orders(user_id);
109 CREATE INDEX idx_orders_status ON orders(status);
110 CREATE INDEX idx_products_category ON products(category_id);

```

4 API Integration & Centralized Exception Handling

The back-end service exposes standard HTTP JSON interfaces. All operational errors must be formatted using a centralized structured schema.

4.1 Centralized Error Schema Representation

If an API request fails, the server must intercept the exception using a `@ControllerAdvice` global class and return the structured JSON payload detailed below. The error response must carry an `X-Correlation-ID` tracking header.

```
1 {
2   "timestamp": "2026-05-22T13:49:00Z",
3   "code": "CONFLICT_STOCK",
4   "message": "Required quantities are not available in catalog inventory.",
5   "correlationId": "err_5f8a9c2d1b",
6   "details": ["Requested item ID 1 stock level is 3, requested 5."]
7 }
```

4.2 API Routes Dictionary

Table 1: API Endpoint Routes Matrix

Category	HTTP Verb	Endpoint Route	Required Role
Authentication	POST	/api/auth/register	Public
Authentication	POST	/api/auth/verify-otp	Public
Authentication	POST	/api/auth/login	Public
Authentication	POST	/api/auth/refresh	Public (HttpOnly cookie)
Product Catalog	GET	/api/products	Public
Product Catalog	GET	/api/products/{id}	Public
Product Catalog	POST	/api/products/{id}/reviews	CUSTOMER (Verified)
Checkout Services	POST	/api/orders/checkout	CUSTOMER (Verified)
Orders Tracking	GET	/api/orders/user/{id}	CUSTOMER / ADMIN
Orders Tracking	POST	/api/orders/{id}/cancel	CUSTOMER
Orders Tracking	POST	/api/orders/{id}/return	CUSTOMER
Admin Dispatch	GET	/api/admin/orders	ADMIN
Admin Dispatch	PUT	/api/admin/orders/{id}/status	ADMIN
Admin Dispatch	PUT	/api/admin/orders/{id}/approve-return	ADMIN

5 UI/UX Design Tokens & Animation Guidelines

To establish a premium, luxury menswear retail presentation across all devices, the seven standalone HTML modules must align with these visual specifications.

5.1 Design Tokens & Color Palette

- **Primary Background elements:** Sophisticated off-black/zinc-900 panels (#18181B) paired with rich deep charcoal backgrounds (#09090B).
- **Text Elements:** Alabaster / slate light shades (#F4F4F5) for body elements, contrast off-white (#FFFFFF) for major heading labels.
- **Accent / Focus Controls:** Antique amber/brass (#B45309) for branding highlights, buy controls, and rating states.
- **Success States:** Emerald (#047857) for paid badges and order delivery steps.

5.2 Mobile-First Responsiveness & WCAG 2.1 AA Checklist

1. **Responsive Grid Fluidity:** All UI containers must resize dynamically using Tailwind layout wraps (`w-full max-w-7xl`). Avoid setting fixed pixel widths. Multi-column structures must reflow to single-column blocks on viewports under 768px wide.
2. **Touch target Sizes:** To facilitate mobile usage, all active buttons, sizing cards, color buttons, and navigation elements must maintain a touch footprint of at least 44px x 44px.
3. **Color Contrast:** Set foreground-to-background contrast parameters to at least 4.5:1 to ensure optimal text legibility.
4. **Accessible Screen Reading:** Embed programmatic alerts (`aria-live="polite"`) around alerts or toaster notifications. Implement keyboard focus trapping inside modals to prevent focus from escaping active boundaries.

5.3 Valid CSS Keyframes & Premium Animations

Custom transitions must be included in a shared stylesheet. They should use the following valid CSS definitions:

```
1 <style>
2 @keyframes modalFadeIn {
3   from { opacity: 0; transform: scale(0.95) translateY(10px); }
4   to   { opacity: 1; transform: scale(1) translateY(0); }
5 }
6
7 @keyframes overlayFadeIn {
8   from { opacity: 0; backdrop-filter: blur(0px); }
9   to   { opacity: 1; backdrop-filter: blur(4px); }
10 }
11
12 .animate-modal {
13   animation: modalFadeIn 0.3s cubic-bezier(0.16, 1, 0.3, 1) forwards;
14 }
15
16 .animate-overlay {
17   animation: overlayFadeIn 0.25s ease-out forwards;
```

```
18 }  
19 </style>
```

6 Observability, CI/CD Gating & Operations

To maintain high availability and quickly resolve issues in production, the application must run within a structured observability pipeline.

6.1 Prometheus Metrics & Alerting Targets

The system must expose metrics to a Prometheus scrape target, triggering operational alerts under the following conditions:

- **Checkout Failure Rate:** Triggers an alert if checkout transaction failures exceed 5% of total traffic within a rolling 15-minute window.
- **Negative Stock Attempts:** Triggers an alert if more than 10 checkout attempts are blocked due to negative stock levels within a single minute (indicating a synchronization mismatch or checkout run).
- **OTP Abuse Rate:** Triggers an alert if more than 10 failed OTP validation attempts originate from a single IP address within an hour.

6.2 CI/CD Compilation Gating Checks

The Maven build pipeline must execute automated tests before deployment, including the following code validation unit test.

```
1 package com.mensdressshop.security;
2
3 import org.junit.jupiter.api.Test;
4 import org.springframework.beans.factory.annotation.Value;
5 import org.springframework.test.context.ActiveProfiles;
6 import org.springframework.boot.test.context.SpringBootTest;
7 import static org.junit.jupiter.api.Assertions.*;
8
9 @SpringBootTest
10 @ActiveProfiles("prod")
11 class DebugOtpGuardTest {
12
13     @Value("${app.security.debug-otp-enabled:false}")
14     private boolean debugOtpEnabled;
15
16     @Test
17     void assertDebugOtpDisabledInProduction() {
18         assertFalse(debugOtpEnabled,
19             "CRITICAL SECURITY FAILURE: debug_otp must be strictly disabled
20             under non-development profiles!");
21     }
22 }
```

6.3 Operations Deployment Checklist

- **Environment Variable Management:** Database credentials, JWT signing private keys, and Razorpay API tokens must be referenced via secure environment variables (ENV) and never hardcoded in files.
- **Database Migrations:** All schema structural transformations must be managed using automated migration scripts (Flyway or Liquibase) kept in version control.

- **Secure SSL/TLS Termination:** All traffic must be encrypted over HTTPS, with insecure HTTP requests automatically upgraded.